Technical Documentation for the Financial Forecasting Module

System Analysis and Design Course

Contents

Introduction 3

2. Data 3

2.1 Data Source and Structure 3

2.2 Exploratory Data Analysis 4

2.2.1 NetBalance Time-Series 4

2.2.2 NetBalance Histogram 4

2.2.3 Autocorrelation Function (ACF) 5

3. System architecture 6

4. Feature Engineering 7

4.1 Calendar Features 7

4.2 Lag Features 8

4.3 Preprocessing Pipeline 8

5. Model Evaluation 9

5.1 Validation Approach 9

5.2 Results of Baseline Models 9

5.3 Learning Curves of Ensemble Models 10

5.4 Learning Curve Gap Comparison 10

5.5 Test-Set R^2 Comparison 12

5.6 Test-Set MAE and RMSE comparison 12

5.7 Cross Validation Standard Deviation 13

6 Result Summary and Model Selection 15

6.1 Overall Comparison Table 15

6.2 Why Random Forest? 15

6.3 Hyperparameter Search Space 16

7. Forecasting Engine 16

8. Retraining Strategy 17
9. System Limitations 17
9.1 Random Nature of Financial Behavior 17
9.2 Limited Feature Space 18
9.3 Synthetic Data 18
9.4 Error Propagation **in** Recursive Forecasting 18
9.5 Limited Data During the Initial Stage 18

Introduction

This report presents the technical documentation for the machine learning-based forecasting module of the AvandCetner project. The objective of the forecasting module is to estimate user's daily net balance over the next seven days based on their transaction history. This report covers the dataset structure, the feature engineering process, validation methodology, experimental results, and a comprehensive analysis of the evaluated models. The main goal of this project is to design and implement a scientifically sound evaluation framework, provide clear technical documentation, and explicitly describe the system's limitations.

Target Variable: Net Balance = Income – Expense
Prediction Horizon: Next 7 days
Validation Approach: TimeSeriesSplit (5-fold cross-validation)

2. Data

2.1 Data Source and Structure

The data used for this stage were generated synthetically to simulate realistic user financial behavior. The dataset was designed to reflect common transaction patterns, including everyday purchases, recurring payments (such as rent), and random financial events.

Statistical analyses of the distribution, autocorrelation, and variability suggest that the synthetic dataset captures real-world financial behavior. However, synthetic dataset cannot fully reproduce financial behavior; These limitations are discussed in Section 7.

2.2 Exploratory Data Analysis

2.2.1 NetBalance Time-Series

The figure illustrates the daily variation in NetBalance over the entire observation period. The time series exhibits high variability and a nonlinear pattern.

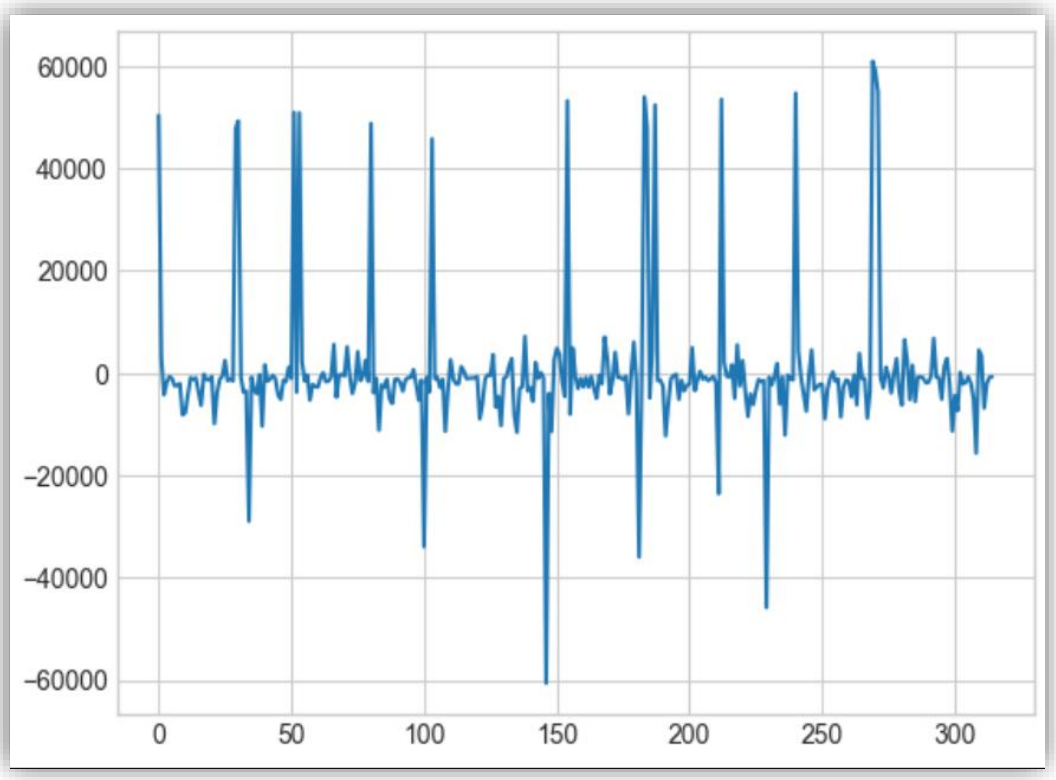


Figure 2.2-1 – NetBalance time series.

The presence of periodic spikes with amplitudes of up to ±60,000 units, together with relatively small day-to-day fluctuations, suggests a bimodal distribution in the data. This pattern is typically caused by large recurring transactions, such as salary payments or rent, occurring alongside ordinary daily transactions.

2.2.2 NetBalance Histogram

The histogram suggests a heavy-tailed, left-skewed distribution.

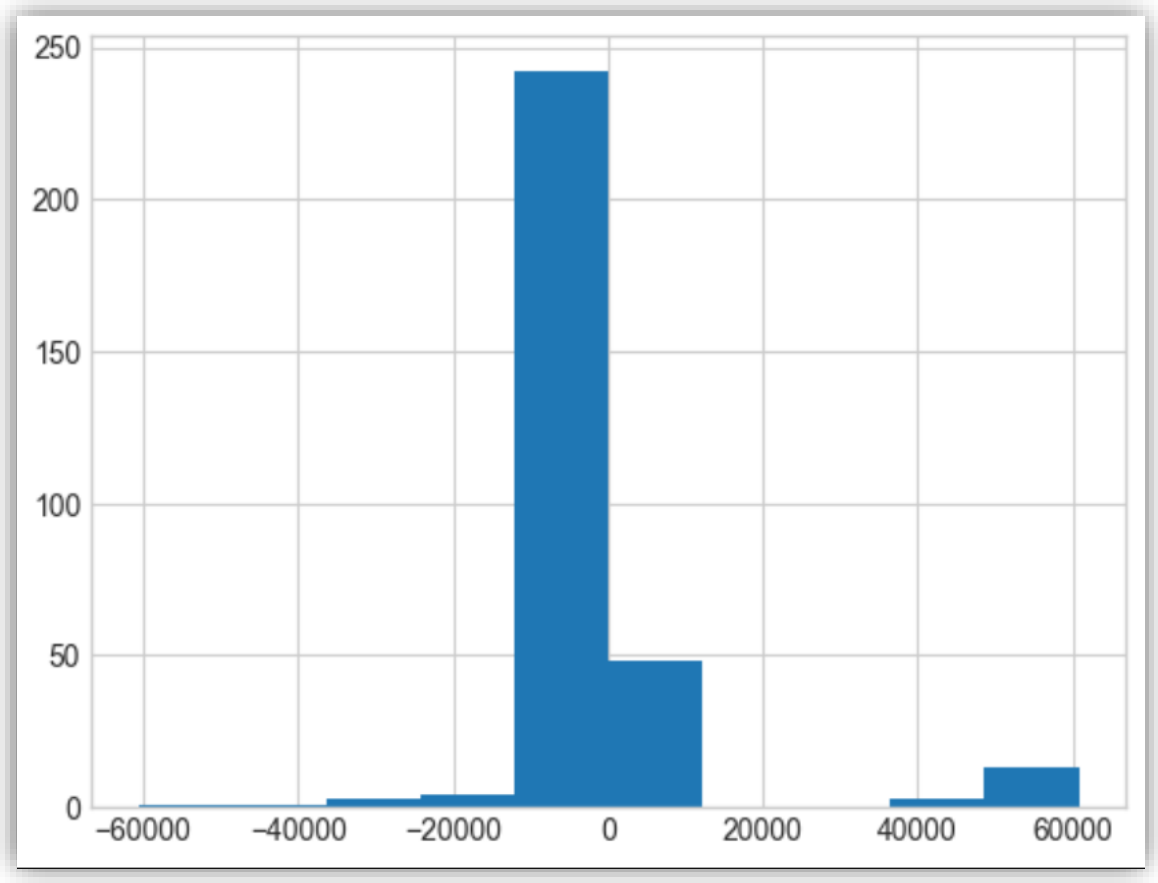


Figure 2.2.2-1 – NetBalance histogram.

The majority of daily values fall within the range of [-10000, +10000]. However, the long right tail (extending to +60,000) suggests that high-value income transactions occur infrequently but involve large amounts. This distributional characteristic makes classical mean-based regression models less suitable for handling these extreme values.

2.2.3 Autocorrelation Function (ACF)

The ACF plot measures the temporal correlation of NetBalance across different lags.

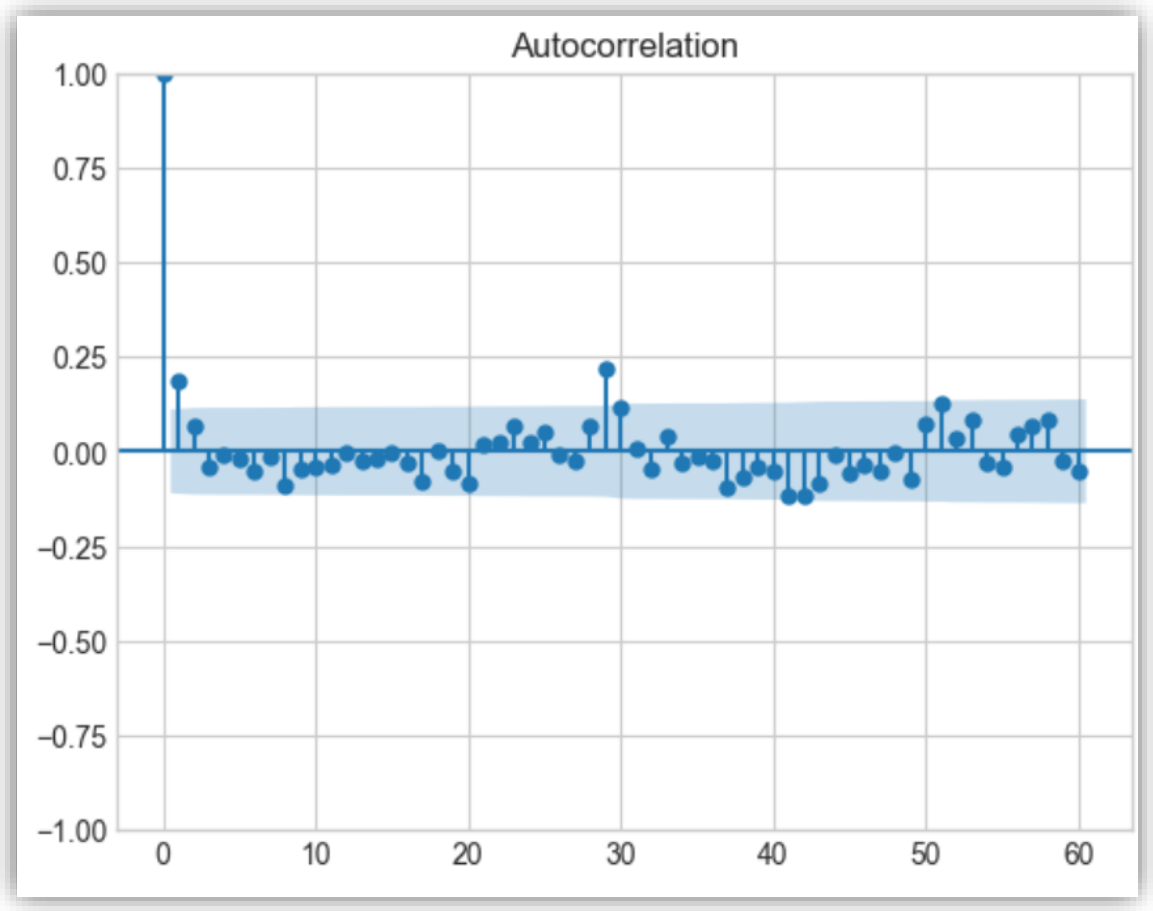


Figure 2.2.3-1 – NetBalance autocorrelation.

The ACF plot suggests that there is no significant temporal correlation between NetBalance at time t and its previous values, except for lag 1, which is slightly above the significance threshold. This observation is consistent with the random nature of day-to-day human spending and explains why models that rely primarily on temporal patterns achieve low R^2 scores: the predictive signal in the data is weak.

3. System architecture

The system has a modular architecture. Every module has a specific responsibility. Dependencies are defined in a way that allows each module to be developed and tested independently.

User eligibility checking: Eligibility/ → eligibility.py

Time and lag feature engineering: Feature_Engineering/ → EDA.py

Preprocessing pipeline: Feature_Engineering/ → pipelines.py

Generating the feature matrix (X) and target vector (y): Feature_Engineering/ → x_y_creation.py

Hyperparameter optimization and model training: Training/ → model_opt.py

Forecasting engine: Prediction/ → predict.py

Retraining manager: Retraining/ retrain.py

4. Feature Engineering

4.1 Calendar Features

Calendar features are derived from the Date column to capture recurring temporal patterns in financial behavior. These features are computed in EDA.py.

Feature	**Type**	**Description**
DayOfWeek	Continuous (0-6)	Captures day-of-week patterns (e.g., Saturday or Sunday)
IsWeekend	Binary (0-1)	Indicates whether the day falls on a weekend, when financial behavior often differs
DayOfMonth	Continuous (1-31)	Captures recurring monthly events such as salary payments or rent
Month	Continuous (1-12)	Captures seasonal patterns throughout the year
WeekOfMonth	Continuous (1-5)	Captures recurring weekly patterns within a month

Table 4.1-1 – Calendar Features

4.2 Lag Features

Lag and rolling window features transform historical information into the feature space. These features are constructed using a one-step shift (lag 1) to prevent future information from leaking into the model (data leakage).

Feature	**Description**
NetBalance_lag1	Represents the previous day's NetBalance
NetBalance_lag7	Captures weekly patterns in NetBalance using the value from seven days earlier
NetBalance_rollmean7	Captures 7-day rolling window mean and estimates the baseline level for NetBalance
NetBalance_rollstd7	Captures the 7-day rolling window standard deviation and estimates recent variability in NetBalance

Table 4.2-1 – Lag Features

Total number of features: 9

['DayOfWeek', 'IsWeekend', 'DayOfMonth', 'Month', 'WeekOfMonth', 'NetBalance_lag1', 'NetBalance_lag7', 'NetBalance_rollmean7', 'NetBalance_rollstd7']

4.3 Preprocessing Pipeline

Preprocessing is managed through a scikit-learn pipeline to ensure that all steps are applied in the correct order without data leakage.

Step 1 – datetime_features: FunctionTransformer → add_date_features()

Step 2 – lag_features: FunctionTransformer → add_lag_features()

Step 3 – finalize: FunctionTransformer → finalize_features()

Step 4 – scale: StandardScaler()

The StandardScaler is applied in the final step to normalize features with different scales (such as DayOfMonth with a range of 1-31 and NetBalance_lag1 with values in thousands). Applying the scaler after generating lag features ensures that the mean and the standard deviation are computed only from the training data.

5. Model Evaluation

5.1 Validation Approach

TimeSeriesSplit with five folds is used because of the time-series nature of the dataset. Unlike standard K-Fold cross-validation, it preserves the chronological order and prevents future information from being used during training.

Validation: TimeSeriesSplit – n_splits = 5

Optimization: RandomizedSearchCV – n_iter = 25 – scoring = neg_RMSE

Evaluation Metrics: R^2, MAE, RMSE, STD of CV

Model Selection Criteria: Learning Curve Gap + STD (The final model was selected by minimizing the sum of the learning-curve gap and the cross-validation standard deviation.)

5.2 Results of Baseline Models

Three linear models were evaluated as the baseline model before evaluating ensemble models. This comparison provides a baseline for assessing the improvement achieved by more complex models.

Model baseline	**R^2 Train**	**R^2 Test**	**MAE Train**	**MAE Test**	RMSE Train	RMSE Test
Linear Regression	0.1375	0.0339	7353.6627	7521.9799	12626.214	13143.265
Ridge	0.1375	0.035	7349.2106	7511.3273	12626.2395	13136.2669
Lasso	0.1375	0.034	7353.4608	7521.4888	12626.2141	13142.9832

Table 5.2-1 – Comparison of Baseline Models

All linear models achieved approximately the same R^2 value (≈0.14) on the training set. However, test R^2 drops to approximately 0.03–0.04. This gap suggests that linear models have limited generalization ability and the assumption of a linear relationship between the features and the target variable does not hold for this dataset.

5.3 Learning Curves of Ensemble Models

Four ensemble models were evaluated: Decision Tree, Random Forest, Gradient Boosting, and Extra Trees. The learning curve shows the relationship between the training set size and both training and validation errors.

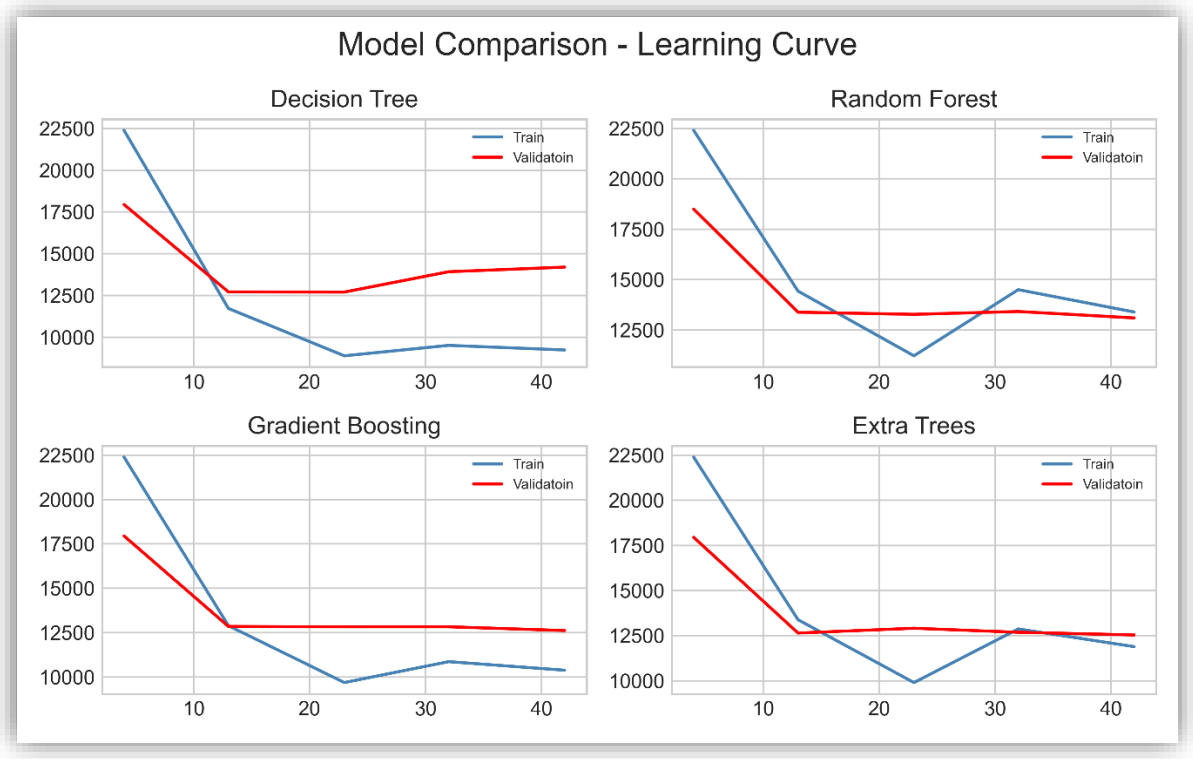


Figure 5.3-1 – Learning curves of the evaluated ensemble models. The x-axis represents the number of training samples, while the y-axis shows the RMSE.

In most models, the training and validation errors converge as the training set size increases. Random Forest and Extra Trees exhibit a smaller gap between the training and validation values for larger training sets, indicating better generalization than Decision Tree. In contrast, Decision Tree exhibits more pronounced overfitting when trained on smaller datasets.

5.4 Learning Curve Gap Comparison

The primary indicator of overfitting is the gap between the training and validation RMSE values. The models with a smaller gap generalize better to unseen data.

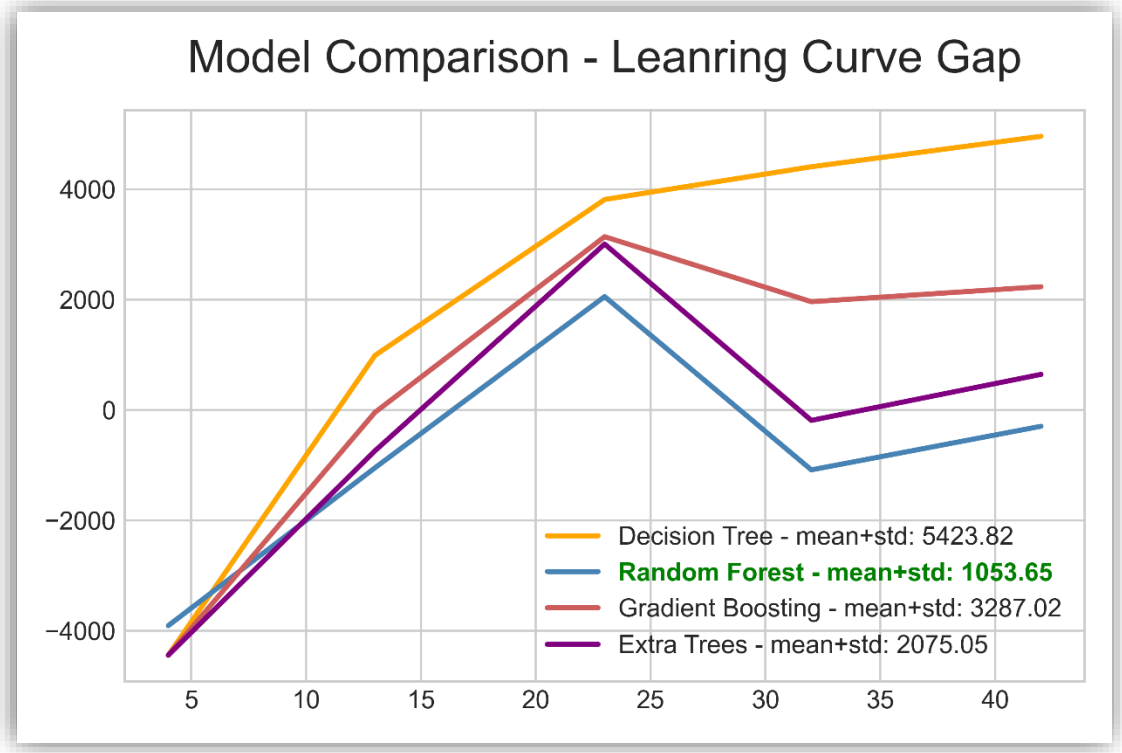


Figure 5.4-1 – Learning Curve Gap Comparison

Random Forest with mean+std = 1053.65, has the smallest gap among the evaluated models. In contrast, Decision Tree with mean+std = 5423.82 exhibits the highest degree of overfitting. Gradient Boosting (3287.02) and Extra Trees (2075.05) lie between these two extremes. The final model was selected by minimizing the sum of the learning-curve gap and the cross-validation standard deviation. This criterion considers both overfitting and model stability simultaneously.

5.5 Test-Set R^2 Comparison



Figure 5.5-1 – R^2 % Comparison on the Test Dataset.

Random Forest with an R^2 of 17.36%, explains the largest proportion of variance. Extra Trees (16.08%) and Gradient Boosting (14.05%) ranked second and third, respectively. Decision Tree with an R^2 of 10.40% has the weakest performance on the test dataset.

An R^2 of 0.17 indicates that the available features explain approximately 17% of the variance in NetBalance. Given the inherently unpredictable nature of human financial behavior, weak autocorrelation observed in Figure 2.2.3-1, and the limited available features in the transaction records, the result is consistent with expectations.

5.6 Test-Set MAE and RMSE comparison

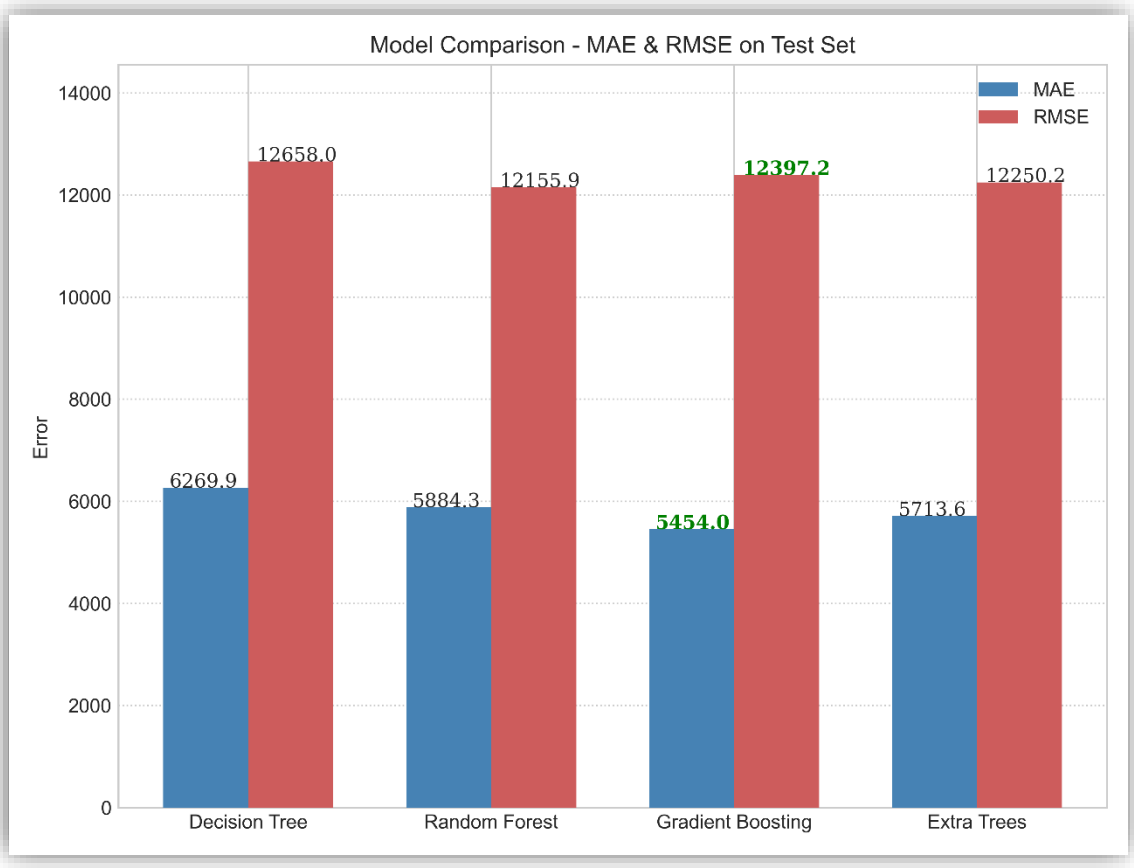


Figure 5.6-1 – The comparison of MAE and RMSE on Test Set

Gradient Boosting achieved the smallest MAE (5454.0). Nevertheless, Random Forest achieved the smallest RMSE (12155.9). This difference highlights the importance of choosing the appropriate model selection criterion:

- MAE: Mean absolute error – Less sensitive to outliers.
- RMSE: Root mean squared error – Weights more on large outliers.

Given large spikes in the data observed in Figure 2.2.1-1, a large RMSE is expected for these models and usually reflects exceptional events rather than day-to-day transactions.

5.7 Cross Validation Standard Deviation

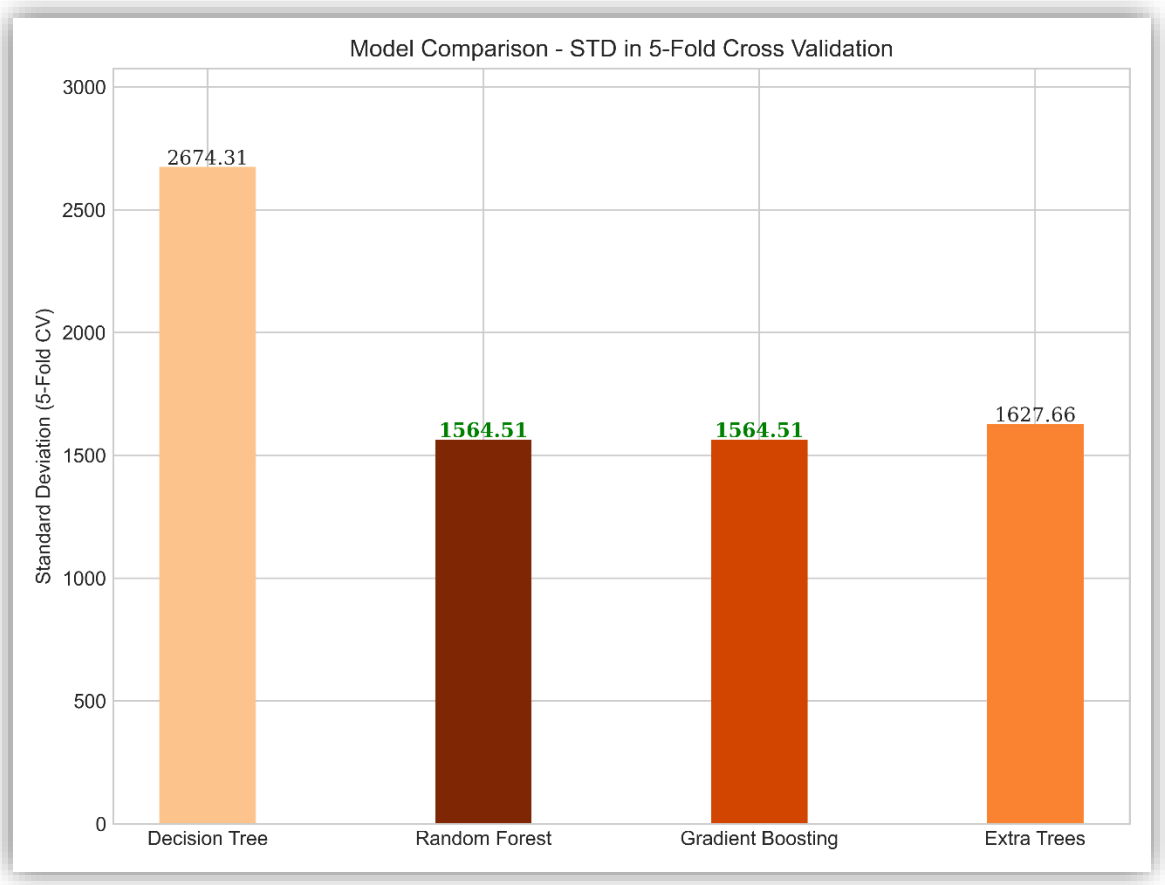


Figure 5.7-1 – Standard Deviation in 5-Fold Cross Validation

Both Random Forest and Gradient Boosting, with a standard deviation of 1564.51, show the lowest cross-validation variability. Extra Trees follows closely with a standard deviation of 1627.66. In contrast, Decision Tree exhibits the highest variation across the cross-validation folds, with a standard deviation of 2674.31.

A lower standard deviation indicates more consistent model performance across different time periods. This property is particularly important in production systems that require periodic retraining, where prediction performance should remain consistent and reliable over time.

6 Result Summary and Model Selection

6.1 Overall Comparison Table

Metric	Decision Tree	Random Forest	Gradient Boosting	Extra Trees
R^2 (Test %)	10.40	**17.36 ✓**	14.05	16.08
MAE (Test)	6,269.9	5,884.3	**5,454.0 ✓**	5,713.6
RMSE (Test)	12,658.0	**12,155.9 ✓**	12,397.2	12,250.2
CV STD	2,674.31	**1,564.51 ✓**	1,564.51 ✓	1,627.66
Learning Curve Gap	5,423.82	**1,053.65 ✓**	3,287.02	2,075.05
CV RMSE Mean	13,159.5	12,743.4	12,715.8 ✓	**12,579.8**

Table 6.1-1 – Overall comparison of the evaluated Models. ✓ indicates the best value in each row.

6.2 Why Random Forest?

Although Gradient Boosting performs slightly better in terms of MAE and CV RMSE mean, Random Forest was selected as the final model for the following reasons:

4. Highest test-set R^2 (17.36%)
5. Smallest learning-curve gap indicates the lowest degree of overfitting
6. Lowest RMSE on the test set
7. High cross-validation stability (STD = 1564.51 equal to Gradient Boosting)
8. Faster training and inference, making it more suitable for periodic retraining

From an engineering perspective, models with a lower level of overfitting generally exhibit more stable performance. As a result, Random Forest is expected to perform more reliably on previously unseen data in production environments.

6.3 Hyperparameter Search Space

RandomForestRegressor(

n_estimators ∈ {100, 200, 300},

max_depth ∈ {3, 5, 7, None},

min_samples_split ∈ {2, 5, 10},

min_samples_leaf ∈ {1, 2, 5},

max_features ∈ {"sqrt", 0.5, None},

random_state=42

)

Search: RandomizedSearchCV – n_iter = 25 – cv = TimeSeriesSplit(n_splits=5) – scoring = neg_RMSE

7. Forecasting Engine

Forecasting is performed using a recursive multi-step strategy. At each inference step, the model predicts the next day's NetBalance, and the predicted value is used as the input for the following step. This process repeats seven times.

Step 1: Historical data → Feature generation → Predict day *t+1*

Step 2: Historical data + Prediction of day *t+1* → Feature generation → Predict day *t+2*

(Repeat until day 7.)

Output: A list of seven predicted daily values + the sum of predicted values

An inherent limitation of the recursive approach is that prediction errors accumulate at each step. Consequently, predictions for later forecast horizons are generally less accurate than those for earlier ones. This phenomenon is known as error propagation in recursive multi-step strategies.

8. Retraining Strategy

RetrainManager manages conditions for retraining. Retraining is performed only when both of the following conditions are satisfied.

Condition	Threshold	Logic
New Transactions	MIN_NEW_TRANSACTION_COUNT=10 At least ten transactions	The model is retrained only when sufficient new data exists
Time Interval	MIN_DAYS_AFTER_LAST_TRAIN=7 At least seven days	Prevents frequent retraining over short time intervals

Table 8-1 – Retraining Conditions

This approach helps avoid unnecessary retraining while ensuring sufficient new information is available. Retraining on duplicate or insufficient new data may lead to overfitting or instability.

9. System Limitations

The following limitations are well-known engineering constraints commonly encountered in financial time-series forecasting systems with limited data.

9.1 Random Nature of Financial Behavior

Human financial behavior is inherently random. Impulsive purchases, unexpected expenses, and events are not completely predictable in any statistical model. The ACF observed in Figure 2.2.3-1 shows that there is no significant temporal correlation in the user's daily NetBalance.

9.2 Limited Feature Space

Existing features are generated from NetBalance history and calendar information, and external explanatory variables such as the inflation rate, holiday events, and salary payment cycle are not available.

9.3 Synthetic Data

The model is trained on synthetic data. Although the performed statistical analyses suggest that synthetic data provide a reasonable approximation of real financial behavior, the model's real-world performance can only be accurately assessed after deployment using real user data. Model performance is expected to improve through periodic retraining.

9.4 Error Propagation **in** Recursive Forecasting

In a recursive approach, in each step prediction error propagates to subsequent steps. Consequently, predictions for later forecast horizons are less accurate than earlier days.

9.5 Limited Data During the Initial Stage

Only a limited amount of training data is available for newly eligible users who have just satisfied the minimum activation criteria (14 days of activity and 20 recorded transactions). As more transaction data become available, model performance is expected to improve gradually.